

Python Basics

Variables, Types, Operators & First Programs

■ LEARNING OUTCOME

By the end of this module you will write and run Python programs, work with all built-in data types, use operators, format strings professionally, and understand Python's design philosophy.

■ Skills You Will Gain

- Install Python and run scripts from terminal
- Use variables, numbers, strings, booleans
- Apply all arithmetic, comparison, and logical operators
- Format strings with f-strings and .format()
- Use the print() and input() functions
- Understand Python's dynamic typing system

■ Python is the world's most popular programming language. It powers data science, machine learning, web development, automation, and scripting. One language -- infinite possibilities.

SECTION 1

Python Basics

Python Philosophy -- The Zen of Python

Beautiful is better than ugly. Explicit is better than implicit. Simple is better than complex. Readability counts. There should be one obvious way to do it. Python uses INDENTATION (4 spaces) to define code blocks -- no curly braces. Comments start with # (inline) or use triple quotes for docstrings.

```
# Python runs line by line -- no compilation needed print("Hello, World!") # output to console
print("Python", "is", "fun") # multiple args separated by space print("Line 1\nLine 2") # \n =
newline character print("-" * 40) # repeat string 40 times # Variables -- no type declaration
needed name = "Alice" age = 28 height = 1.65 is_active = True no_value = None # Multiple assignment
x, y, z = 10, 20, 30 a = b = c = 0 # all point to same value first, *rest = [1,2,3,4,5] # starred
unpacking # Dynamic typing -- type can change value = 42 print(type(value)) # <class 'int'> value =
"hello" print(type(value)) # <class 'str'> # User input name = input("Enter your name: ") # always
returns str age = int(input("Enter age: ")) # convert to int num = float(input("Enter number: "))
```

SECTION 2

Data Types

Type	Example	Description
int	42, -7, 0	Whole numbers, unlimited size in Python
float	3.14, -0.5, 1e10	Decimal numbers (IEEE 754 double precision)
str	'hello', "world"	Text -- immutable sequence of Unicode characters
bool	True, False	Boolean -- subclass of int (True=1, False=0)
None	None	Represents absence of value -- like null in other languages
complex	3+4j	Complex numbers with real and imaginary parts
list	[1,'a',True]	Ordered, mutable sequence of any items
tuple	(1,'a',True)	Ordered, IMMUTABLE sequence
dict	{'key':'val'}	Key-value mapping -- unordered in Python < 3.7, ordered in 3.7+
set	{1,2,3}	Unordered collection of UNIQUE items

```
# Numbers x = 1_000_000 # underscore separator for readability bi = 0b1010 # binary: 10 ox = 0xFF #
hex: 255 oc = 0o17 # octal: 15 # Arithmetic operators print(10 + 3) # 13 addition print(10 - 3) # 7
subtraction print(10 * 3) # 30 multiplication print(10 / 3) # 3.333... true division (always float)
print(10 // 3) # 3 floor division (integer result) print(10 % 3) # 1 modulo (remainder) print(10 **
3) # 1000 exponentiation # Strings name = "Alice" greeting = 'Hello' multi = """Line 1 Line 2 Line
3""" # String operations print(name.upper()) # ALICE print(name.lower()) # alice
print(name.startswith("A")) # True print(len(name)) # 5 print(name[0]) # A (indexing)
print(name[-1]) # e (negative index from end) print(name[1:3]) # li (slicing) print(name[::-1]) #
ecilA (reverse) # f-strings (Python 3.6+) -- preferred age = 28; height = 1.65 print(f"Name:
{name}, Age: {age}") print(f"Height: {height:.2f} m") # 2 decimal places print(f"Pi:
{3.14159:.4f}") # 3.1416 print(f"Score: {95/100:.1%}") # 95.0% print(f"{age:>10}") # right-align in
```

```
10 chars print(f"{1000000:,.}") # 1,000,000 with comma separator
```

SECTION 3

Operators and Type Conversion

```
# Comparison operators (return bool) print(5 == 5) # True print(5 != 4) # True print(5 > 3) # True
print(5 >= 5) # True print("a" < "b") # True (lexicographic) # Logical operators print(True and
False) # False print(True or False) # True print(not True) # False # Python-specific operators
print(5 is 5) # True (same object in memory) print([] is []) # False (different objects) print(1 in
[1,2,3]) # True (membership test) print("lo" in "hello") # True (substring check) # Augmented
assignment x = 10 x += 5 # x = 15 x -= 3 # x = 12 x *= 2 # x = 24 x /= 5 # x = 4 x **= 3 # x = 64 #
Type conversion print(int("42")) # 42 print(int(3.9)) # 3 (truncates, no rounding)
print(float("3.14")) # 3.14 print(str(42)) # "42" print(bool(0)) # False print(bool("")) # False
print(bool([])) # False print(bool(None)) # False print(bool("hello")) # True (any non-empty
string) print(bool(1)) # True
```

■ PRACTICE Q & A

Q1. What is the difference between / and // in Python?

A: / always returns a float (true division): 10/3 = 3.333. // returns the floor integer (floor division): 10//3 = 3. Always use // when you need an integer result.

Q2. What is None in Python and how is it different from False?

A: None represents the absence of a value (like null). It is its own type (NoneType). bool(None) is False, but None is not the same as False -- use 'is None' to check, not '== None'.

Q3. What is the difference between == and is?

A: == compares VALUES (are they equal?). is compares IDENTITY (are they the same object in memory?). Always use == for value comparison. Use 'is' only for None: 'if x is None'.

Q4. What are f-strings and why are they preferred?

A: f-strings (f'Hello {name}') are string literals with embedded expressions evaluated at runtime. Preferred because: readable, concise, support formatting specs, and faster than % or .format() methods.

Q5. What does Python's dynamic typing mean?

A: Variables have no fixed type -- the type is determined by the value assigned. The same variable can hold an int, then a string, then a list. Python checks types at runtime, not at compile time.

■ Python Basics Cheat Sheet	
<code>print(f'{name} is {age}')</code>	f-string -- preferred formatting
<code>type(value)</code>	Check type of variable
<code>int(x) / float(x) / str(x)</code>	Type conversion
<code>x, y = 10, 20</code>	Multiple assignment
<code>first, *rest = lst</code>	Starred unpacking
<code>10 // 3 = 3</code>	Floor division
<code>10 % 3 = 1</code>	Modulo / remainder
<code>10 ** 3 = 1000</code>	Exponentiation
<code>'a' in 'abc'</code>	Membership / substring test
<code>x is None</code>	Identity check (use for None)
<code>name[1:4]</code>	String slicing
<code>name[::-1]</code>	Reverse string